Algorithms for genomic data analysis: *k*-mer spectral methods

Aleksander Jankowski

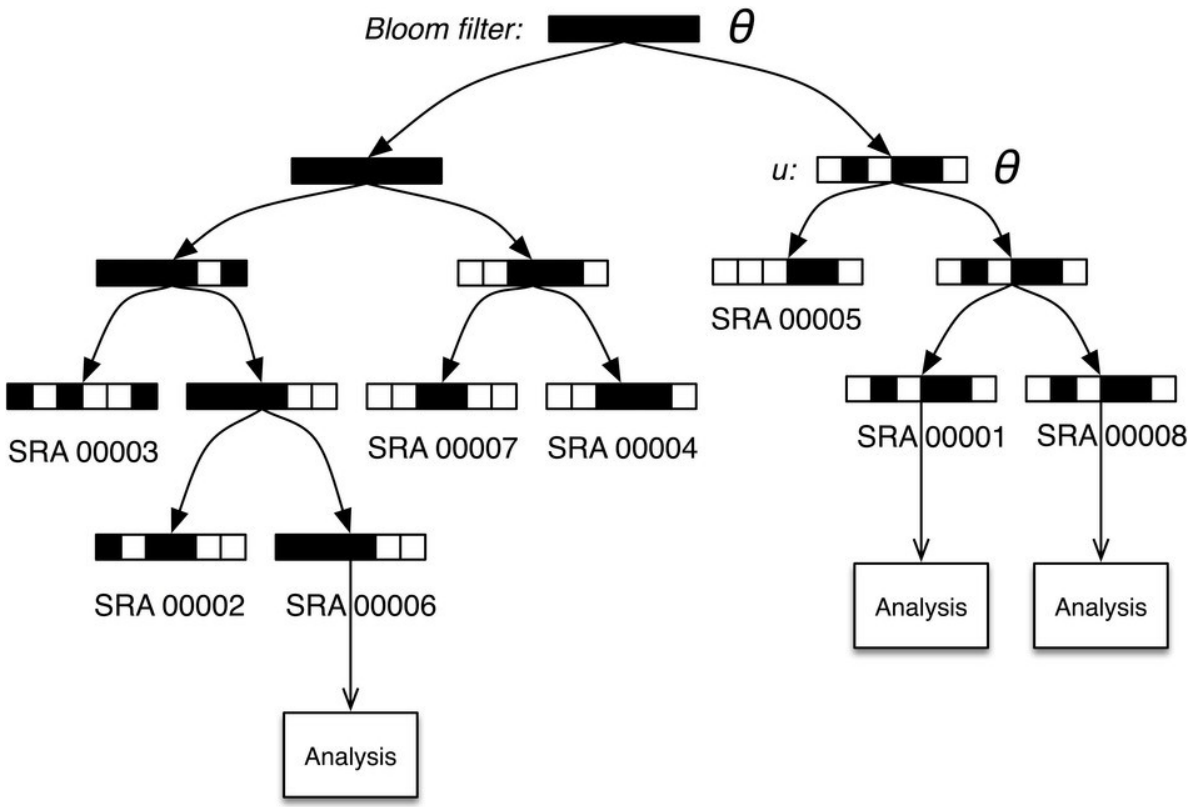
November 12, 2024

UNIVERSITY
OF WARSAW

Outline

- Sequence Bloom Trees – indexing large collections of short-read experiments quickly using a hierarchy of Bloom filters
- gkm-SVM: gapped *k*-mer Support Vector Machine

Schematic of a Sequence Bloom Tree



Solomon and Kingsford, Nature Biotechnology 2016.

The parameter θ governs a query's tolerance to errors.

Sequence Bloom Trees: querying

- Given a query sequence q and a Sequence Bloom Tree T, the sequencing experiments that contain q can be found by breaking q into its constituent set of k-mers K_q and then flowing these k-mers over T (starting from the root).
- At each node u, the Bloom filter $b(u)$ at that node is queried for each of the k-mers in K_q. If more than $\theta|K_q|$ of the k-mers are reported to be present in $b(u)$, the search proceeds to all of the children of u, where θ is a cutoff between 0 and 1 governing the stringency required of the match.
- Ignoring the effects of sequence boundaries, a general SBT query with N k-mers and k-mer size k tolerates at least $N(1-\theta)/k$ k-mer mismatches between the query and the stored data.

Sequence Bloom Trees: construction

- Given a (possibly empty) Sequence Bloom Tree T, a new sequencing experiment s can be inserted into T by first computing the Bloom filter $b(s)$ of the k-mers present in s and then walking from the root along a path to the leaves and inserting s at the bottom of T.

Sequence Bloom Trees: construction

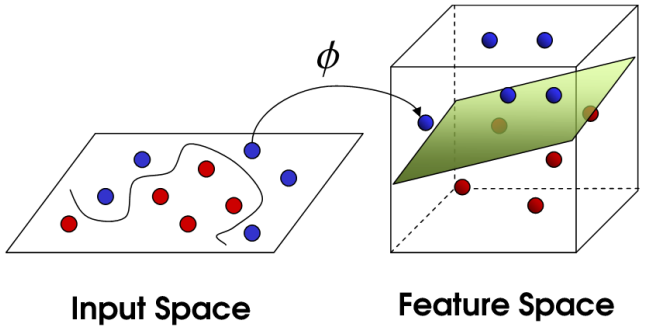
- Inserting s at the bottom of T goes as follows:
  - When at node u, if u has a single child, a node representing s (and containing $b(s)$) is inserted as u's second child.
  - If u has two children, $b(s)$ is compared against the Bloom filters $b(\text{left}(u))$ and $b(\text{right}(u))$ of the children of u. The child with the more similar filter under the Hamming distance between the filters becomes the current node.
  - If u has no children, a new union node v is created as a child of u's parent. This new node has two children: u and a new node representing s.
- Each filter consists of a bit vector of length m and a set of h hash functions $h_i : U \to [0, m)$ that map items to bits in the bit vector.

Gapped *k*-mers: motivation

- Previous approach (*k*-mer-SVM) uses combinations of short (6-8 bp) *k*-mer frequencies to predict the activity of DNA regulatory sequences.
- Transcription Factor Binding Sites (TFBS) typically are 6-20 bp long, and thus cannot be completely represented by the short *k*-mers.
- Limitation: longer *k*-mers generate extremely sparse feature vectors (i.e. most *k*-mers do not appear in training sequences, or appear only once), which causes overfitting problems.
- To overcome these limitations, new method (gkm-SVM) is proposed, using as features a full set of *k*-mers with gaps.

Support Vector Machines and kernel function

- At the heart of Support Vector Machines is the *kernel trick*, mapping the inputs into high-dimensional feature space.



https://www.jeremyjordan.me/support-vector-machines/

- The high-dimensional space is used only implicitly – it suffices to know the kernel function, which calculates the similarity between any two elements in the input space.
- Our set of features (gapped k-mers) is characterized by
  1. l, the whole word length including gaps, and
  2. k, the number of informative, or non-gapped, positions in each word. The number of gaps is thus $l - k$.

Kernel function a.k.a. similarity score calculation

- For i-th gapped k-mer (characterized by l and k), let y_i^S – the counts of the corresponding gapped k-mers appearing in the sequence S
- The similarity score, or a kernel function, between two sequences S_1 and S_2 is defined as the normalized inner product of the corresponding feature vectors y^{S_1} and y^{S_2}:

$$K(S_1, S_2) = \frac{\langle y^{S_1}, y^{S_2} \rangle}{||y^{S_1}|| \cdot ||y^{S_2}||}$$

  where $\langle y^{S_1}, y^{S_2} \rangle = \sum_i y_i^{S_1} \cdot y_i^{S_2}$ and $||y^S|| = \sqrt{\langle y^S, y^S \rangle}$.
- The similarity score $K(S_1, S_2)$ is always between 0 and 1, and $K(S, S)$ is equal to 1.

A more efficient similarity score calculation

- It is not necessary to compute all possible gapped k-mer counts. It suffices to consider only the full l-mers present in the two sequences – they contribute to the similarity score via all gapped k-mers derived from them.

A more efficient similarity score calculation

- A more compact sum is as follows:

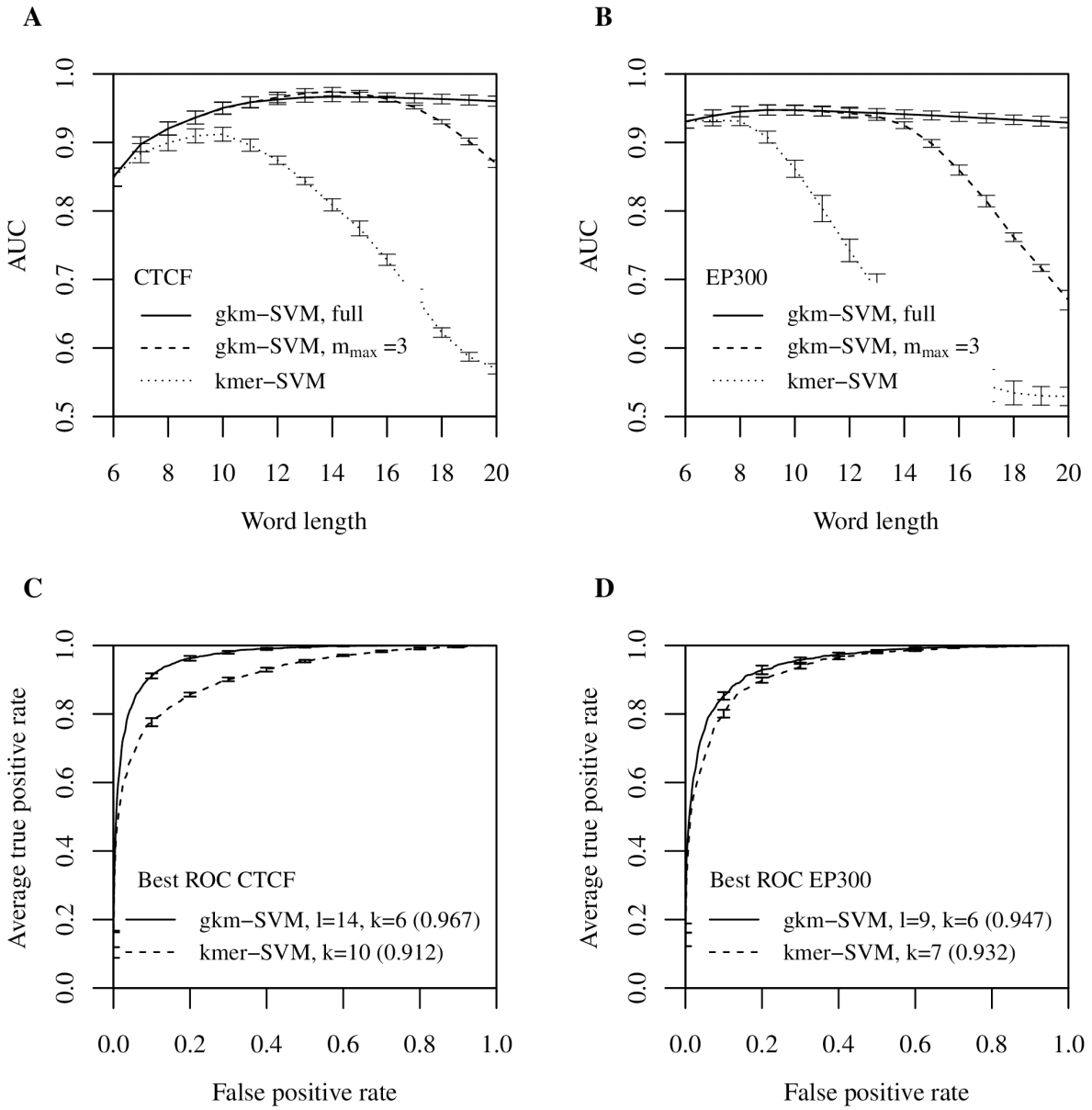
$$\langle y^{S_1}, y^{S_2} \rangle = \sum_i \sum_j h_{lk}(u_i^{S_1}, u_j^{S_2})$$

  where $u_i^{S_1}$ is the i-th l-mer appearing in S_1 and $u_j^{S_2}$ is the j-th l-mer appearing in S_2.

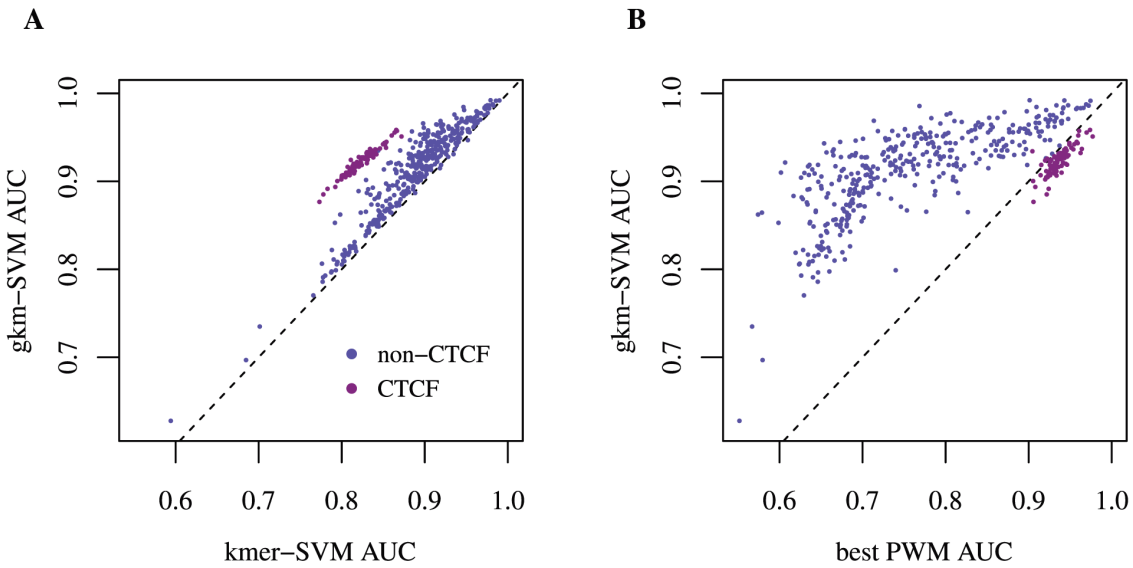
- The coefficient $h_{lk}(u_1, u_2)$ only depends on the number of mismatches m between the two full l-mers (u_1 and u_2). Since each l-mer pair with m mismatches contributes to $\binom{l-m}{k}$ common gapped k-mers, the coefficient $h_{lk}(m)$ is given by:

$$h_{lk}(m) = \begin{cases} \binom{l-m}{k} & l - m \geq k \\ 0 & \text{otherwise.} \end{cases}$$

gkm-SVM outperforms kmer-SVM over a wide range of *k*-mer lengths



gkm-SVM outperforms kmer-SVM and the best known Position Weight Matrix on ENCODE ChIP-seq data

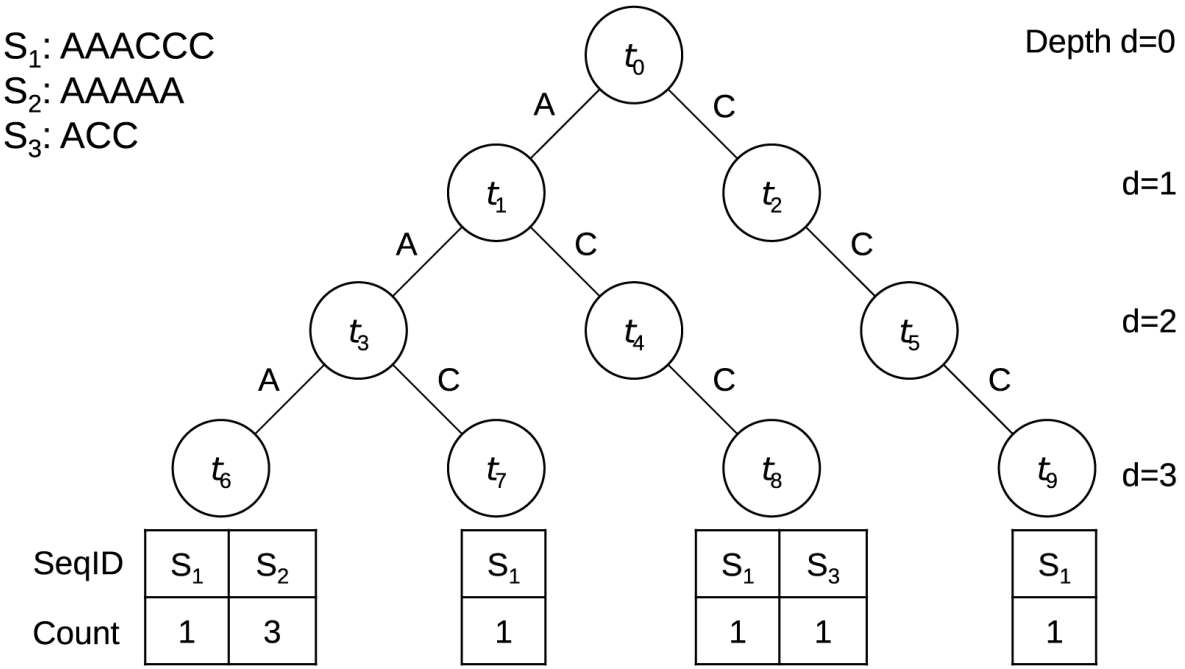


Ghandi et al., PLOS Computational Biology 2014.

Fast computation of mismatch profiles using k-mer tree structure

- We use a k-mer tree to hold all the l-mers in the collection of all of the sequences.
- Construction: add a path for every l-mer observed in a training sequence.
- Each node t_i at depth d represents a sub-sequence of length d, denoted by $s(t_i)$, which is determined by the path from the root of the tree to the node t_i.
- Each leaf of the tree represents an l-mer, and holds the list of training sequence labels in which that l-mer appeared and the number of times that l-mer appeared in each sequence.

Using the *k*-mer tree structure




```
DFS(node, depth, {(node, mismatches)})
DFS(t0 , 0, {(t0, 0)})
    DFS(t1 , 1, {(t1, 0), (t2, 1)})
        DFS(t3 , 2, {(t3, 0), (t4, 1), (t5, 2)})
            DFS(t6 , 3, {(t6, 0), (t7, 1), (t8, 2), (t9, 3)})
            DFS(t7 , 3, {(t6, 1), (t7, 0), (t8, 1), (t9, 2)})
        …
    DFS(t2 , 1, {(t1, 1), (t2, 0)})
        …
```

Ghandi et al., PLOS Computational Biology 2014.

Using the k-mer tree structure

- To evaluate the mismatch profile we traverse the tree in a depth-first search (DFS) order.
- For each node t_i, at depth d, we store the list of pointers to all the nodes t_j at depth d for which $s(t_i)$ and $s(t_j)$ have at most $l - k$ number of mismatches. We also store the number of mismatches between $s(t_i)$ and $s(t_j)$.
- When reaching a leaf node, we increment the corresponding mismatch profile $N_m(S_i, S_j)$ for each pair of sequences S_i in that leaf node's sequence list, and all the S_j's in the list of sequences in the pointer list for that leaf node.
- At the end of one DFS traversal of the tree, the mismatch profiles for all pairs of sequences are completely determined.

k-mer tree structure: further speedup

- We can also introduce an optional parameter $m_{\max}$, which limits the maximum number of mismatches.
- By setting $m_{\max}$ smaller than $l - k$, we only consider l-mer pairs that have at most $m_{\max}$ number of mismatches.
- This can reduce calculation significantly by ignoring l-mer pairs which potentially contribute less to the overall similarity scores.

Bibliography

Brad Solomon and Carl Kingsford.
Fast search of thousands of short-read sequencing experiments.
Nature Biotechnology 34(3): 300–302, 2016.

Mahmoud Ghandi, Dongwon Lee, Morteza Mohammad-Noori and Michael A. Beer.
Enhanced Regulatory Sequence Prediction Using Gapped *k*-mer Features.
PLOS Computational Biology 10(12): e1004035, 2014.